Containerization Fast Track

Day 1: Containers and Docker

United Airlines · Pluralsight instructor-led training

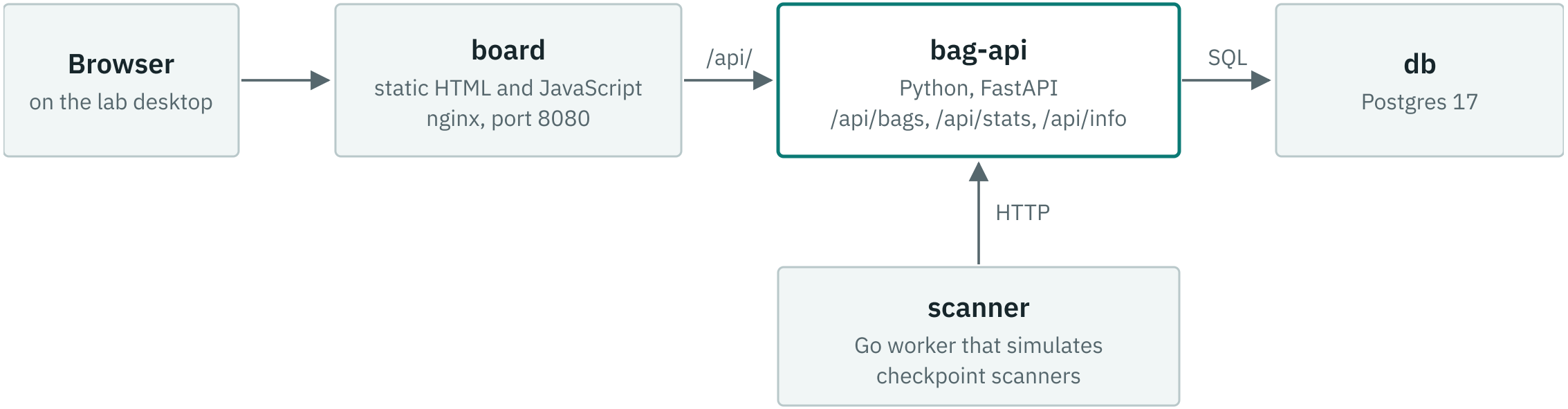
Nick Lotz

Your lab machine and its tools

Tool	What you use it for
Docker Engine 29 with the Compose plugin	Build and run containers
Portainer CE 2.45 LTS	A web UI for Docker on port 9000
Local registry (`registry:3`)	Stores images at `localhost:5000`
Trivy 0.76 and hadolint	Scan images, produce SBOMs, lint Dockerfiles
Harness Open Source 3.3.0	Git repositories and CI pipelines on port 3000
k3d 5.9, k3s v1.36 and kubectl 1.36	A local Kubernetes cluster with three nodes
Headlamp 0.46	A desktop UI for Kubernetes
VS Code	Editing files

Each of you has an Ubuntu 24.04 desktop VM on AWS EC2 with 8 vCPU, 16 GB of memory and an 80 GB disk. You install every tool during the labs and need no AWS, Docker Hub or Harness account.

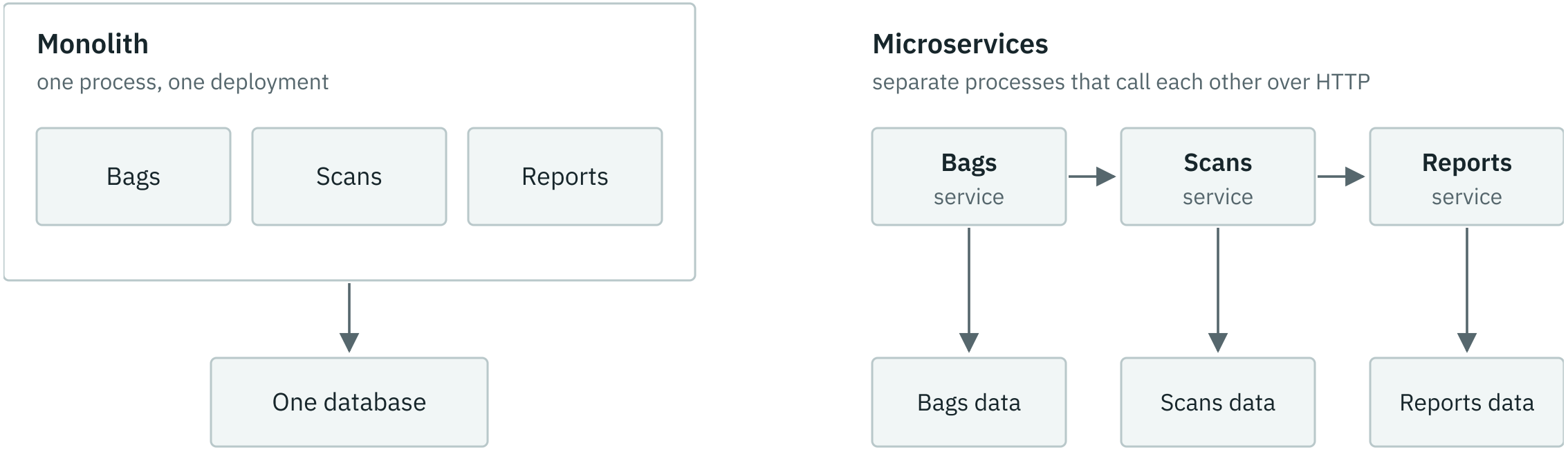
BagTrack, the system you build in this course



Checkpoint scans move each bag through checked_in, security, sorted, loaded, unloaded and claimed.

The board proxies `/api/` requests to bag-api by name. You containerize bag-api in Lab 02 and add Postgres and the board in Lab 03.

Monolithic and microservice architectures



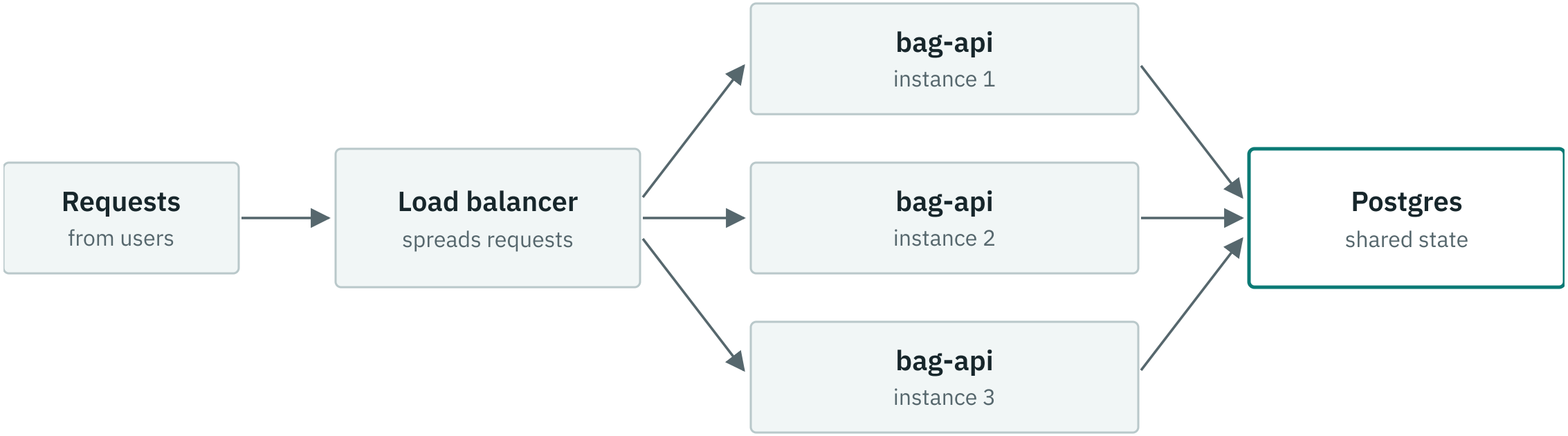
A monolith is built and deployed as one unit. Lewis and Fowler (2014) describe microservices as “a suite of small services, each running in its own process and communicating with lightweight mechanisms”.

Trade-offs between monoliths and microservices

Concern	Monolith	Microservices
Deployment	The whole application at once	Each service on its own schedule
Scaling	Copies of the whole application	Only the services that need more capacity
Failure	Function calls inside one process	Network calls that can be slow or fail
Data	One database with local transactions	Each service owns its data, and consistency across services needs design
Operations	One application to run and monitor	Many services to deploy, trace and secure

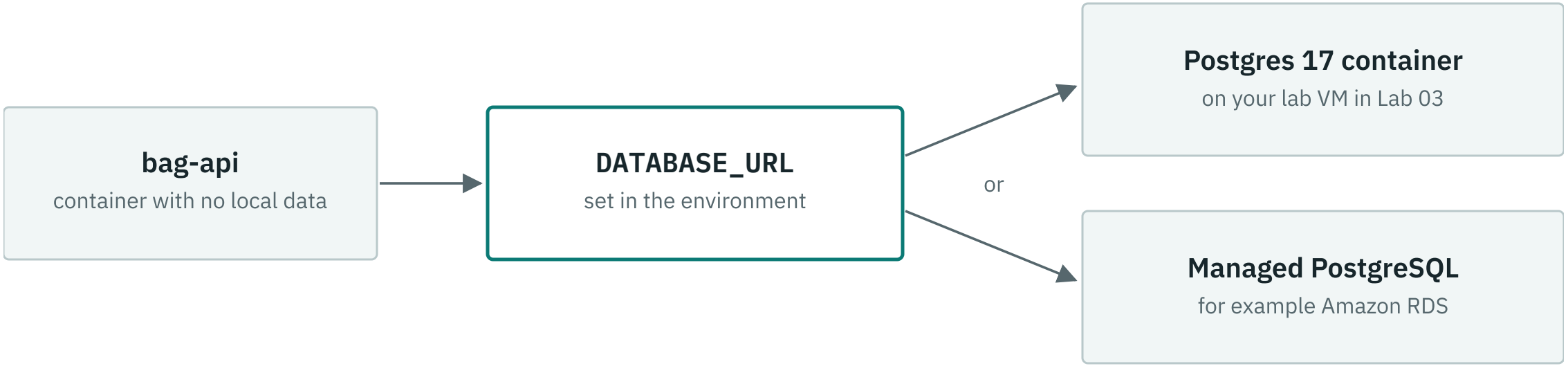
Newman (2021) recommends microservices for a specific goal, such as independent deployment, because they add cost in operations, testing and data consistency.

Stateless services and horizontal scaling



A stateless service keeps no client data in memory or on local disk between requests (Wiggins, 2011). Any instance can answer any request, so you add instances to handle more load.

State belongs in backing services



Twelve-factor treats a database, queue or cache as a backing service, an attached resource that the app finds through a URL in its config (Wiggins, 2011). Changing the URL switches the database without changing the code.

Configuration comes from the environment

- Config is everything that varies between deploys, such as database URLs, credentials and hostnames (Wiggins, 2011)
- The application reads it from environment variables when it starts
- The same image runs in development, test and production
- Secrets are injected at run time and are never built into the image

ECS task definitions, Compose files and Kubernetes Pods all set environment variables for a container.

Disposable processes start fast and stop cleanly

- Start in a few seconds, so new instances can take traffic soon after a scale-out
- On SIGTERM, stop accepting new requests, finish the current ones, then exit
- A worker returns its current job to the queue before it exits
- Expect to be stopped at any time, by a deploy, a scale-in or host maintenance

Wiggins (2011) calls processes that can be started or stopped at a moment's notice disposable. Docker, ECS and Kubernetes all send SIGTERM first and SIGKILL after a grace period.

The CNCF definition of cloud native

- Practices to develop, build and deploy workloads in public, private and hybrid clouds, at scale and in a programmatic, repeatable way
- Loosely coupled systems that are secure, resilient, manageable, sustainable and observable
- Usually built from containers, service meshes, multi-tenancy, microservices, immutable infrastructure, serverless and declarative APIs
- Automation that allows frequent, predictable, high-impact changes with minimal toil

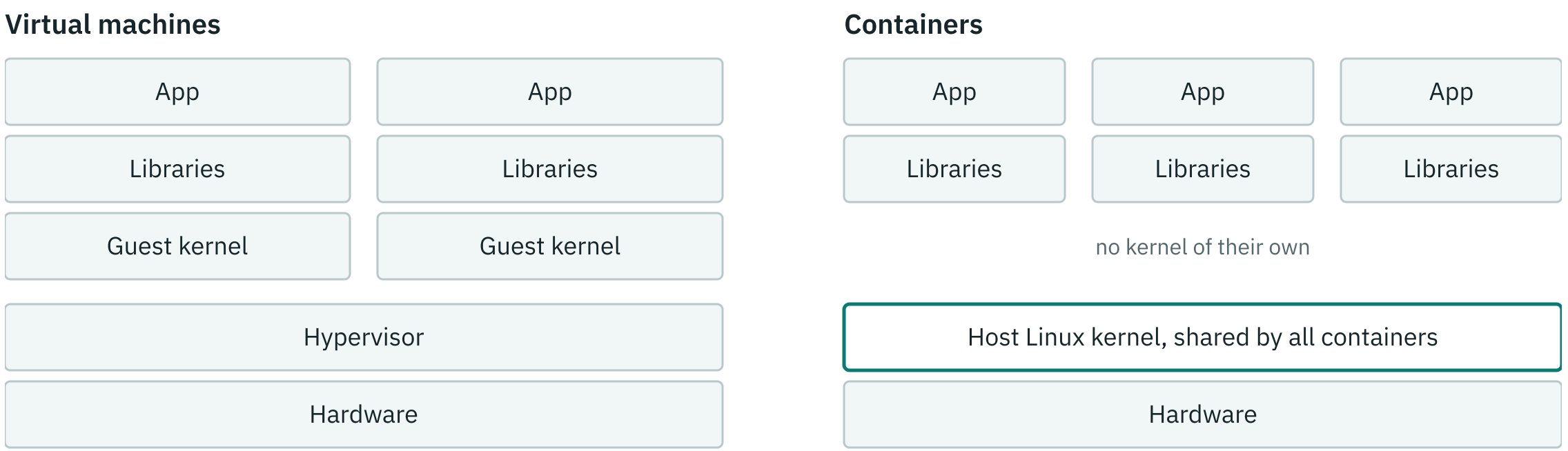
Version 1.1 replaced the 2018 text in 2024 and added multi-tenancy, serverless, security and sustainability (CNCF TOC, 2018, 2024).

Where containers fit at United

At United	What it does	In this course
Amazon ECS with AWS Fargate	Runs most container workloads, with no servers to manage	Docker Compose and ECS task-definition files
Amazon EKS	Managed Kubernetes	A k3d cluster running k3s v1.36
Amazon ECR	Stores container images	A local registry at `localhost:5000`
Harness platform	Runs deployments	Harness Open Source 3.3.0

The Digital Technology teams chose ECS with Fargate in 2019–2020, “when Kubernetes was hard”.

How virtual machines and containers share a host



A virtual machine runs its own kernel on virtual hardware that the hypervisor provides. Containers are processes that share the host's kernel, isolated by namespaces and limited by cgroups.

Comparing virtual machines and containers

Property	Virtual machine	Container
Kernel	Its own guest kernel	The host's kernel
Starts by	Booting an operating system	Starting a process
Image holds	A whole operating system	An application, its libraries and a minimal user space
Isolation	Virtual hardware from a hypervisor	Kernel features: namespaces, cgroups, capabilities and seccomp
Typical size	Gigabytes	Megabytes to a few hundred megabytes

Felter et al. (2015) found that containers performed as well as or better than KVM virtual machines in almost all of their tests. AWS Fargate runs each task in its own isolated virtual environment (Amazon Web Services, n.d.).

A short history of Linux containers

Year	Development
1979	`chroot` in Version 7 Unix gives a process its own root directory
2000	FreeBSD jails isolate processes, users and network addresses
2002	Linux 2.4.19 adds the first namespace type, for mount points
2008	Linux 2.6.24 adds control groups, and LXC combines them with namespaces
2013	Docker adds a portable image format and a simple command line (Merkel, 2014)
2015	Docker, CoreOS and others found the Open Container Initiative

Docker used kernel features that already existed and made them easy to use, with images that run the same way on any Linux host.

Image, container, registry and runtime

Term	Meaning	Example
Image	A read-only package of filesystem layers and settings, such as the start command	`postgres:17`
Container	A running or stopped instance of an image, with its own writable layer	The `db` container in BagTrack
Registry	A server that stores images and serves them over HTTP	Amazon ECR, `localhost:5000`
Runtime	Software that creates and runs containers from images	containerd, runc

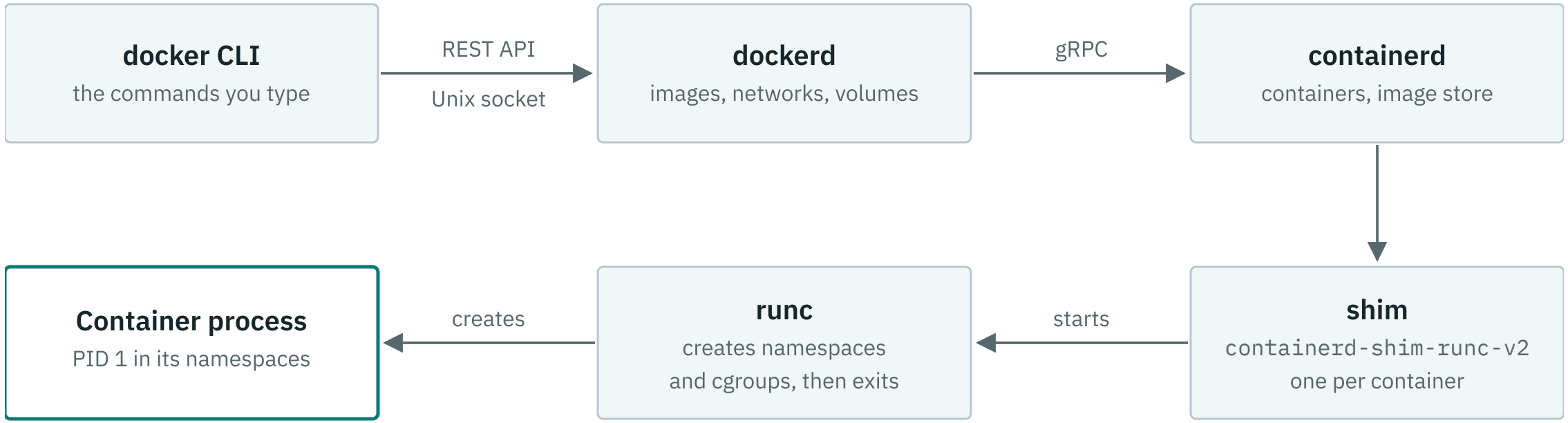
One image can start many containers, in the same way one program file can start many processes.

The three OCI specifications

Specification	What it defines	Implemented by
Image Format v1.1	The manifest, configuration, filesystem layers and image index	BuildKit, Podman, containerd
Runtime v1.2	A bundle with a root filesystem and `config.json`, and the create, start, kill, delete and state operations	runc, crun
Distribution v1.1	An HTTP API to push, pull and list content in a registry	Docker Hub, Amazon ECR, `registry:3`

Because Docker, Podman, containerd and CRI-O all follow these specifications, an image you build on your VM runs under any of them (Open Container Initiative, 2024).

How Docker runs a container



The runc process exits after it starts the container process, and the shim stays as the parent of the container process. Docker 29 also keeps images in containerd's image store by default.

Other tools that run OCI containers

Tool	How it works	Where you meet it
Docker Engine	dockerd on top of containerd and runc	Your lab VM, developer machines
Podman	No daemon, can run without root, accepts most docker commands	Red Hat Enterprise Linux
nerdctl	A docker-style command line for containerd	Hosts with containerd and no Docker
CRI-O	A runtime made only for Kubernetes, through the CRI	Red Hat OpenShift

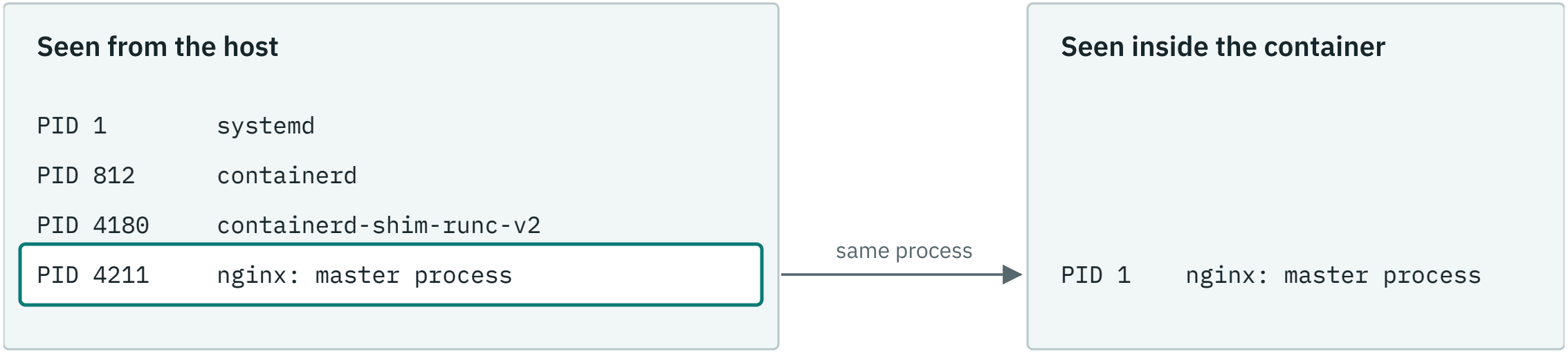
Kubernetes removed its built-in Docker support, dockershim, in release 1.24 (Kubernetes, n.d.). Amazon EKS AMIs use containerd from Kubernetes 1.24 onward (Amazon Web Services, n.d.).

Container registries

Registry	Address	Notes
Docker Hub	`docker.io`, the default	Anonymous limit of 100 pulls per 6 hours per IP
Amazon ECR	`<account>.dkr.ecr.<region>.amazonaws.com`	Private, holds United's images
ECR Public	`public.ecr.aws`	Mirrors Docker Official Images under `docker/library`
Local registry	`localhost:5000`	A `registry:3` container on your VM

All four implement the OCI Distribution API, so docker push and docker pull work the same way with each. The labs pull base images from ECR Public to avoid Docker Hub's limit (Docker, n.d.).

A container is a process on the host



`docker inspect --format '{{.State.Pid}}' web` prints the host PID of a container's main process. The kernel isolates this process with namespaces and limits its resources with cgroups.

Linux namespaces

Namespace	Isolates	In a Docker container
`pid`	Process IDs	The main process is PID 1 and sees only its own processes
`net`	Interfaces, IP addresses, ports, routes	Its own `eth0` and IP address
`mnt`	Mount points	Its own root filesystem, built from the image
`uts`	Hostname and NIS domain name	The hostname defaults to the container ID
`ipc`	System V IPC and POSIX message queues	Its own shared memory
`user`	User and group IDs	Not used unless you enable it
`cgroup`	The view of the cgroup tree	Its own cgroup appears as the root

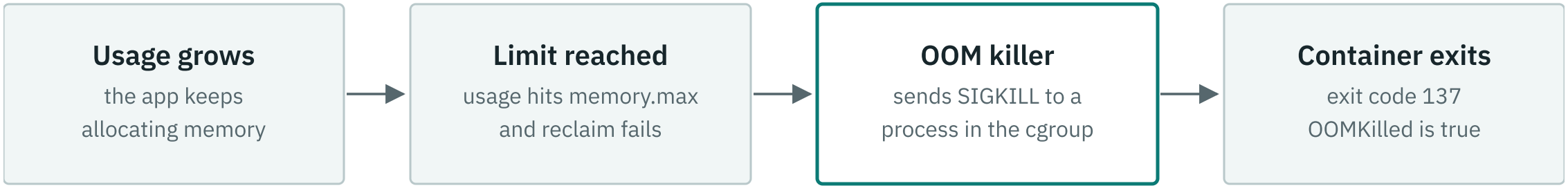
A namespace wraps a global resource so that the processes inside see their own isolated instance of it (Kerrisk, n.d., namespaces(7)).

Control groups limit resources

Resource	docker run flag	cgroup v2 file	At the limit
Memory	`--memory 64m`	`memory.max`	The kernel reclaims memory, then the OOM killer ends a process
CPU	`--cpus 0.5`	`cpu.max`	The process is throttled and runs slower
Processes	`--pids-limit 100`	`pids.max`	New processes and threads fail to start

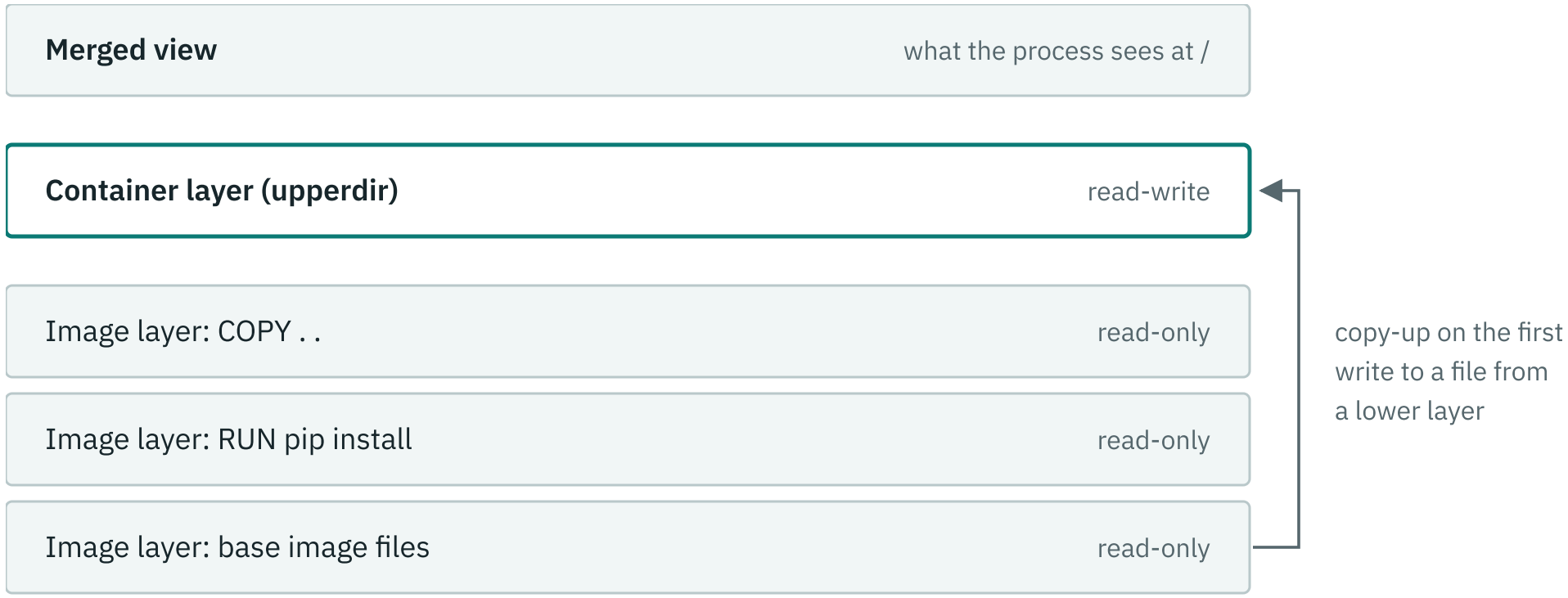
Ubuntu 24.04 uses cgroup v2, with one unified hierarchy under `/sys/fs/cgroup` (Kerrisk, n.d., cgroups(7)). Without these flags a container has no limits and can use as much as the host allows (Docker, n.d.).

What happens when a container runs out of memory



The OOM killer chooses only among the processes in the container's cgroup, so the host and other containers keep running. If it kills the main process, the container stops with exit code 137, which is 128 plus 9 for SIGKILL.

How overlayfs builds a container's filesystem



When a process first changes a file from an image layer, overlayfs copies the whole file into the container layer and changes the copy (Docker, n.d.). `docker diff` lists the files a container has added (A), changed (C) or deleted (D).

Capabilities and seccomp

Capabilities

- Linux splits root's privileges into about 40 capabilities (Kerrisk, n.d., capabilities(7))
- Docker keeps 14 by default, such as `CHOWN` and `NET_BIND_SERVICE`
- `--cap-drop ALL` removes them, and `--cap-add` adds one back

Seccomp

- A filter on the system calls a process can make
- Docker's default profile blocks about 44 of more than 300 system calls (Docker, n.d.)
- Blocked calls include `reboot`, `bpf` and `keyctl`

Both restrictions also apply to root inside the container.

Inspecting a container from the host

```
# Host PID and full ID of the container named web
PID=$(docker inspect --format '{{.State.Pid}}' web)
CID=$(docker inspect --format '{{.Id}}' web)
ps -o pid,ppid,user,cmd -p "$PID"

# Its namespaces, to compare with /proc/self/ns
sudo ls -l /proc/$PID/ns

# Its cgroup and its memory limit
cat /proc/$PID/cgroup
cat /sys/fs/cgroup/system.slice/docker-$CID.scope/memory.max

# Run the host's ip tool inside its network namespace
sudo nsenter --target "$PID" --net ip addr
```

The `/proc` entries and `nsenter` work the same way for containers that containerd starts on a Kubernetes node.

Reading container exit codes

Code	Meaning	Common cause
0	The main process exited successfully	A one-off task finished
1	The application reported an error	An unhandled exception
125	Docker could not run the container	An unknown flag, or a name already in use
126	The command exists but cannot run	The file is not executable
127	The command was not found	A typo in CMD, or a missing binary
137	Killed by SIGKILL (128 + 9)	The OOM killer, docker kill, or the stop timeout
143	Ended by SIGTERM (128 + 15)	docker stop, when the process exits on the signal

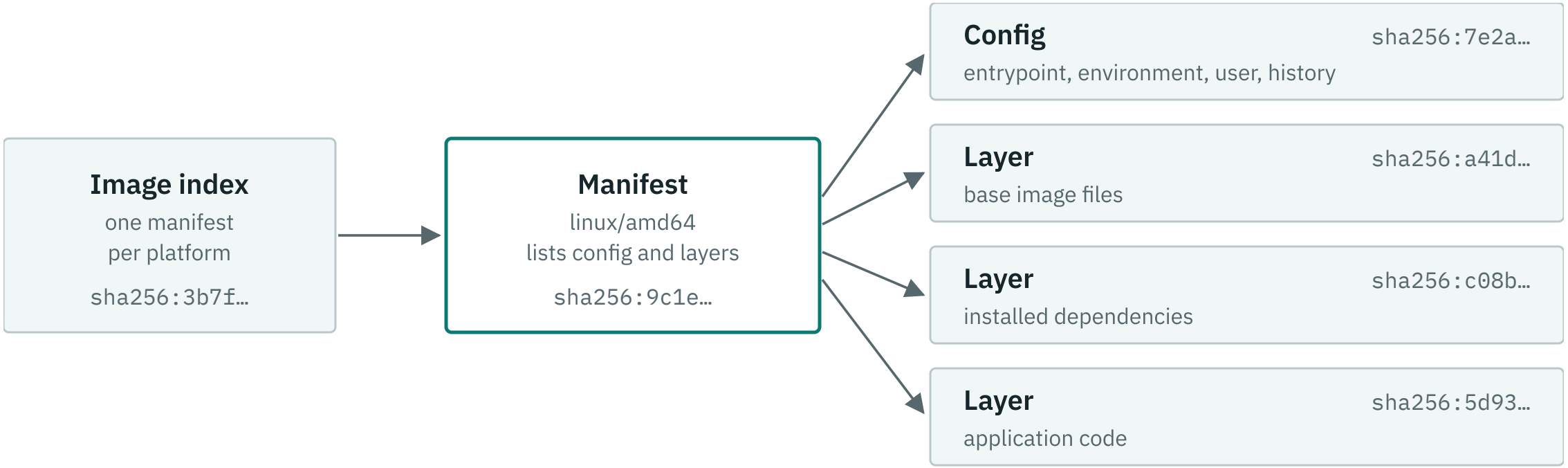
Docker returns 125, 126 and 127 itself, and every other code is the exit status of the container's command (Docker, n.d.).

Lab 01: Your lab machine and first containers

`labs/01-first-containers.md`

- Install Docker Engine 29 from Docker's apt repository
- Run, stop and remove containers, and publish a port
- Start Portainer and look at your containers on port 9000
- Find a container's host PID, namespaces and cgroup limits
- Cause an OOM kill and read the exit codes

How an image is stored



Every part is stored and requested by the SHA-256 digest of its content, which is called content-addressable storage (Open Container Initiative, 2024). Images that share a base layer store and download that layer once.

The main Dockerfile instructions

Instruction	What it does
`FROM`	Sets the base image for this build stage
`WORKDIR`	Sets the working directory for later instructions and for the container
`COPY`	Copies files from the build context into the image
`RUN`	Runs a command during the build and saves the result as a layer
`ENV`	Sets an environment variable in the image
`USER`	Sets the user that later instructions and the container run as
`EXPOSE`	Documents the port the application listens on
`CMD`, `ENTRYPOINT`	Set the command that runs when the container starts

Only RUN, COPY and ADD create filesystem layers. The other instructions change the image configuration, and `docker history` shows them as 0 B steps.

How the build cache works

```
FROM public.ecr.aws/docker/library/python:3.13-slim
WORKDIR /app

# Dependencies change rarely, so they are installed first
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

# Source code changes often, so it is copied last
COPY . .
CMD ["uvicorn", "bagtrack.main:app", "--host", "0.0.0.0", "--port", "8000"]
```

Docker reuses a cached layer while the instruction and the files it copies are unchanged. Once a layer changes, every layer after it is rebuilt, so put the instructions that change least near the top (Docker, n.d.).

The build context and .dockerignore

```
# .dockerignore
.git
.venv/
__pycache__/
*.pyc
.pytest_cache/
.env
```

`docker build` sends the build context, the folder you name, to the builder without the paths that `.dockerignore` lists. Leaving out `.git`, virtual environments and `.env` files makes the context smaller and keeps secrets out of the image.

Exec form, shell form and PID 1

Exec form

```
CMD ["uvicorn", "bagtrack.main:app", "--host", "0.0.0.0"]
```

- Docker starts uvicorn with no shell
- uvicorn is PID 1 and receives the SIGTERM from `docker stop`
- Works in images without a shell

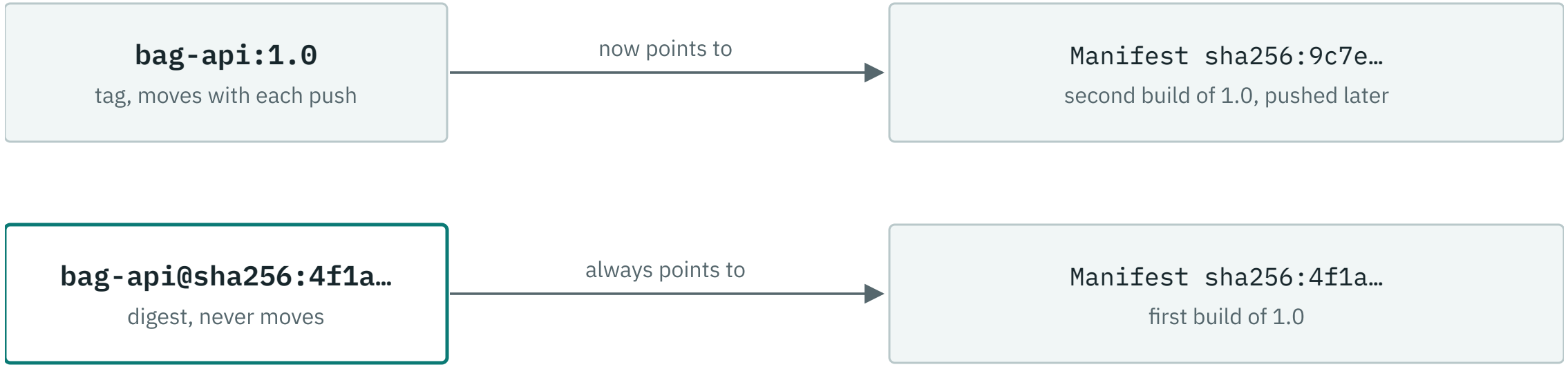
Shell form

```
CMD uvicorn bagtrack.main:app --host 0.0.0.0
```

- Docker wraps the command in `/bin/sh -c`
- The shell may stay as PID 1 and not pass SIGTERM on (Docker, n.d.)
- Needs `/bin/sh` in the image

A process running as PID 1 receives SIGTERM only if it installs a handler for it (Kerrisk, n.d., pid_namespaces(7)).

Tags and digests



A tag is a name that the next push can move to a different manifest. A digest is the SHA-256 hash of the manifest, so it always refers to the same content (Open Container Initiative, 2024).

Consume, extend or build a custom image

Approach	What you do	BagTrack example
Consume	Run a published image as it is, configured at run time	db runs `postgres:17` with a volume and environment variables
Extend	Start FROM an application image and add your files or settings	board adds its HTML and JavaScript to an unprivileged nginx image
Custom	Start FROM a language or minimal base and build your own application	bag-api starts FROM python, and on Day 2 the scanner starts FROM scratch

With a custom image you control every layer, and you must also apply base image updates and security patches yourself.

Pushing and pulling images

```
# The registry host is the first part of the name
docker tag bag-api:1.0 localhost:5000/bag-api:1.0

# Upload the layers the registry lacks, then the manifest
docker push localhost:5000/bag-api:1.0

# List the tags in a repository (OCI Distribution API)
curl http://localhost:5000/v2/bag-api/tags/list

# Download the manifest by tag, then any missing layers
docker pull localhost:5000/bag-api:1.0
```

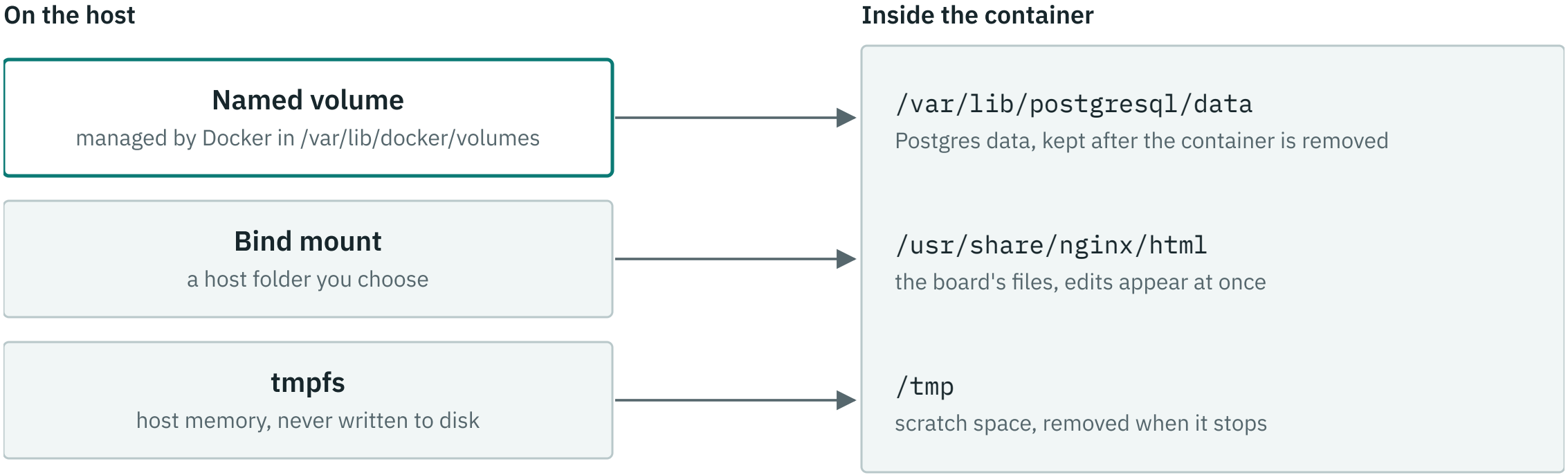
A name without a registry host, such as `python:3.13-slim`, refers to Docker Hub. For Amazon ECR you run `docker login` with a token from the AWS CLI before you push (Amazon Web Services, n.d.).

Lab 02: Build images

`labs/02-images.md`

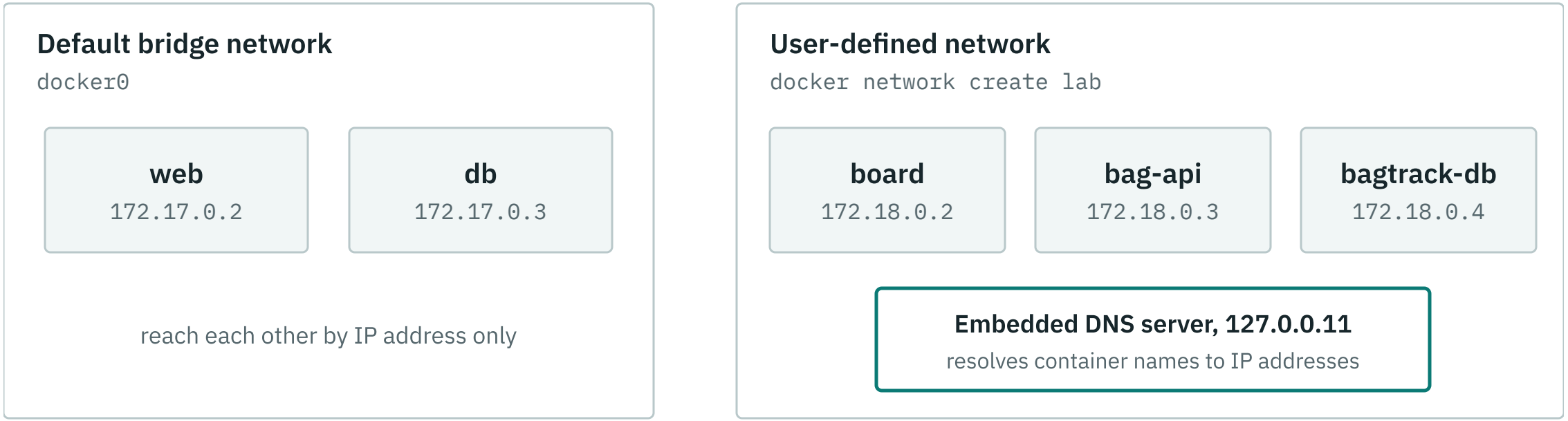
- Pull and inspect a base image, and see the writable layer with `docker diff`
- Write the Dockerfile for bag-api, then build and run it
- Change the code, watch the build cache, and add a `.dockerignore`
- Run a local registry on port 5000 and push your image to it
- Pull the image back by its digest

Volumes, bind mounts and tmpfs



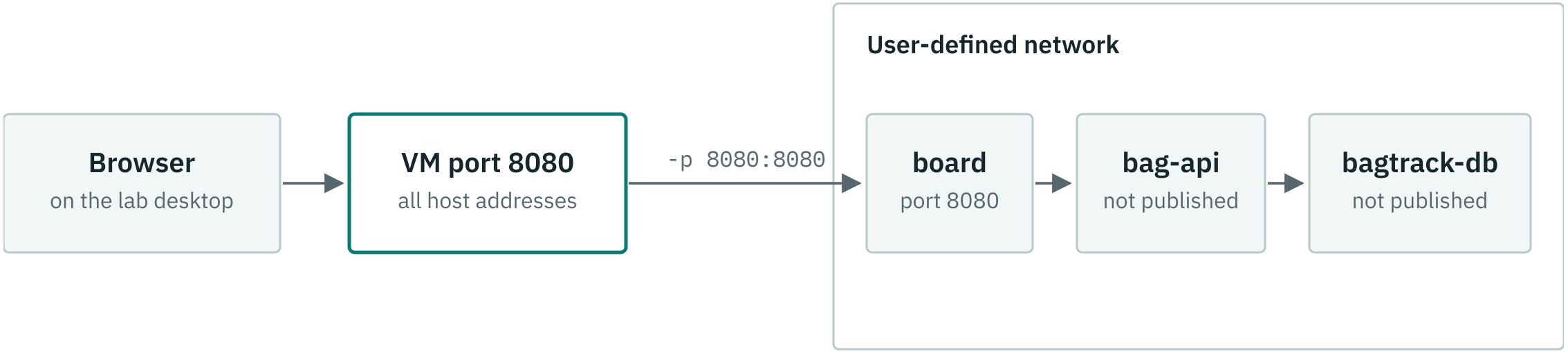
Files written to the container's writable layer are deleted with the container. A named volume keeps data that must outlive a container, such as the Postgres data directory in Lab 03 (Docker, n.d.).

Bridge networks and container DNS



On a user-defined network, Docker's embedded DNS server at 127.0.0.11 resolves container names to their current IP addresses (Docker, n.d.). Containers on the default bridge network can reach each other only by IP address.

Publishing ports



`-p 8080:8080` publishes port 8080 on every host address and forwards it to port 8080 in the container (Docker, n.d.). Publish only what users need, and use `-p 127.0.0.1:8080:8080` to limit a port to the host itself.

Restart policies

Policy	Docker restarts the container
`no`	Never. This is the default.
`on-failure[:N]`	After it exits with a non-zero code, up to N times
`always`	Whenever it stops, and again when the Docker daemon starts
`unless-stopped`	Like `always`, except after you stop it yourself

A restart policy reacts only to the main process exiting. If you stop a container by hand, Docker does not restart it until the daemon restarts or you start it again (Docker, n.d.).

Health checks

```
# Mark Postgres healthy once it accepts connections
docker run -d --name db \
  --health-cmd "pg_isready -h localhost -U postgres" \
  --health-interval 10s --health-retries 3 \
  -e POSTGRES_PASSWORD=example \
  public.ecr.aws/docker/library/postgres:17

# The STATUS column shows (healthy) or (unhealthy)
docker ps

# In a Dockerfile, the same check becomes part of the image
HEALTHCHECK --interval=10s --retries=3 \
  CMD pg_isready -h localhost -U postgres
```

Docker marks a container unhealthy after the set number of failed checks in a row, and it does not restart it. ECS runs only the health check in the task definition and replaces unhealthy tasks in a service (Amazon Web Services, n.d.).

A troubleshooting checklist

Command	What it tells you
`docker ps -a`	Whether the container is running, and its exit code
`docker logs --tail 50 web`	What the application printed before it failed
`docker inspect web`	State, OOMKilled, health, mounts, networks and environment
`docker exec -it web sh`	Files, settings and DNS as the container sees them
`docker events`	What Docker did and when, such as restarts, kills and health changes

Read the status and the logs before you change anything.

Lab 03: Data, networks and health

`labs/03-data-and-networks.md`

- Run Postgres with a named volume, and show that the data survives a new container
- Connect bag-api, Postgres and the board on a user-defined network
- Publish the board's port and bind-mount its files
- Add restart policies and health checks
- Work through the troubleshooting drill with logs, inspect, exec and events

References

Amazon Web Services. (n.d.). *Amazon ECS Developer Guide*; *Amazon ECS API Reference*; *Amazon ECR User Guide*; *Amazon ECR Public User Guide*; *Amazon EKS User Guide*. docs.aws.amazon.com

Cloud Native Computing Foundation Technical Oversight Committee. (2018, 2024). *CNCF Cloud Native Definition* v1.0 and v1.1. github.com/cncf/toc/blob/main/DEFINITION.md

Docker. (n.d.). *Docker documentation*: Dockerfile reference, build cache, storage drivers, volumes, networking, restart policies, resource constraints, seccomp, CLI reference, Docker Hub usage. docs.docker.com

Felter, W., Ferreira, A., Rajamony, R., and Rubio, J. (2015). An updated performance comparison of virtual machines and Linux containers. *IEEE ISPASS 2015*.

Kerrisk, M. (n.d.). *Linux manual pages*: namespaces(7), pid_namespaces(7), cgroups(7), capabilities(7). man7.org

Kubernetes. (n.d.). *Kubernetes documentation*: Container runtimes. kubernetes.io/docs

Lewis, J., and Fowler, M. (2014). *Microservices*. martinfowler.com/articles/microservices.html

Merkel, D. (2014). Docker: lightweight Linux containers for consistent development and deployment. *Linux Journal*, 2014(239).

Newman, S. (2021). *Building Microservices* (2nd ed.). O'Reilly.

Open Container Initiative. (2024). *OCI Image Format Specification* v1.1; *OCI Runtime Specification* v1.2; *OCI Distribution Specification* v1.1. github.com/opencontainers

Wiggins, A. (2011). *The Twelve-Factor App*. 12factor.net